

Trabalho B

Cena Interactiva com Câmaras Fixas, Câmara Móvel, Instanciação de Primitivas Geométricas, Animações Simples e Colisões

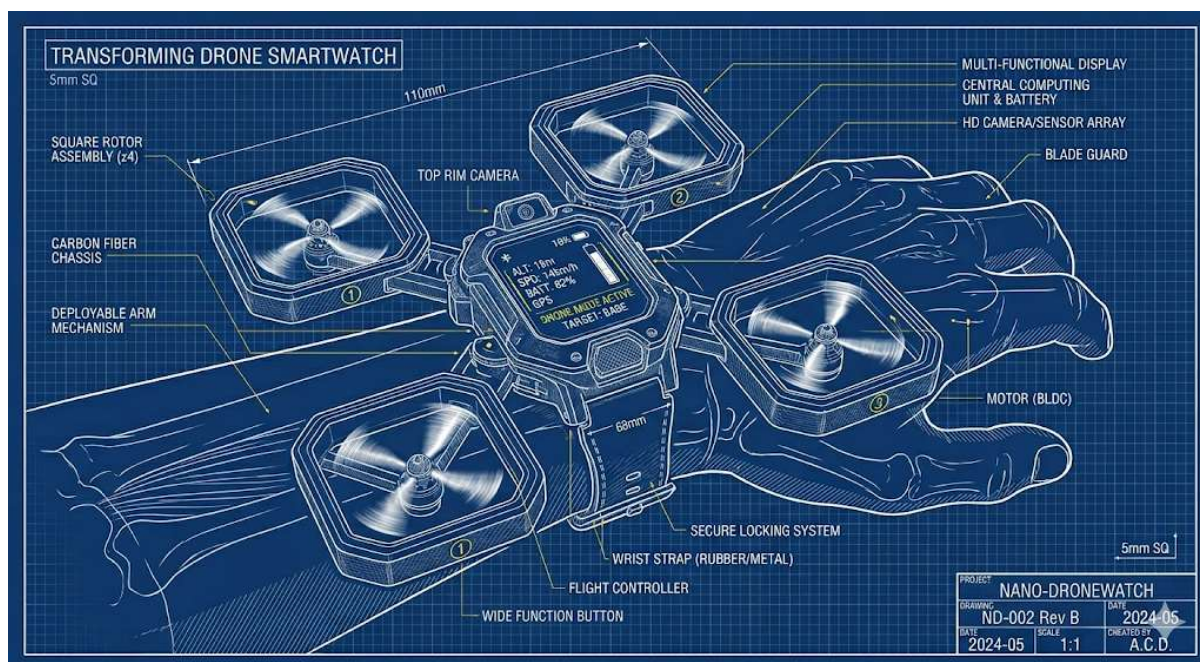


Figura 1 – Ilustração conceptual de um “DroneWatch” evidenciando alguns dos seus elementos constituintes.

Fonte – Imagem gerada por Gemini com a seguinte prompt: “render a concept image of a smartwatch that is also a quadcopter with four rotors; the rotors are square shaped, but you can see the blades; place a camera on the top rim and a wide button on the bottom rim; show the wrist of the user; present a blueprint rendering style”

Este trabalho consiste no desenvolvimento de uma cena gráfica interactiva em *Three.js* centrada no artefacto especulativo *DroneWatch*, um dispositivo fictício que consiste num híbrido entre um *smartwatch* e um *drone* com quatro rotores (*quadcopter*) (Figura 1).

Objectivos

Os objetivos do segundo trabalho de laboratório são: (i) compreender e implementar a arquitectura de uma aplicação gráfica interactiva; (ii) explorar os conceitos básicos de modelação geométrica por instanciação de primitivas; (iii) explorar o conceito de câmara virtual; (iv) perceber as diferenças entre projecção ortogonal e projecção perspectiva; (v) aplicar técnicas básicas de animação; e (vi) compreender e implementar técnicas simples de detecção de colisões.

Todos os grupos devem submeter o código até ao dia **15 de Maio às 23:59h** (final da Semana 4). As discussões serão realizadas nos respectivos turnos na **1ª ou 2ª aula** da Semana 5, entre 18 a 22 de

Maio. O Trabalho B corresponde a **4 valores** da nota da componente laboratorial. A realização deste trabalho tem um esforço estimado de **14 horas por elemento do grupo**, distribuído por **duas semanas**.

Não esquecer de comunicar ao docente do laboratório as **horas despendidas pelo grupo (média do grupo)** na realização deste trabalho.

Lista de Tarefas

1. **[0,25 valores]** Primeiro, definir o fundo da cena com uma cor cinzenta escura. Seguidamente, incluir três câmaras fixas com projecção ortogonal, correspondentes às vistas de topo, lateral e frontal da cena. Estas câmaras devem estar orientadas para o centro da cena. Definir ainda duas câmaras fixas adicionais, posicionadas de modo a captar todos os elementos da cena. Ambas devem ser colocadas no mesmo ponto, fora dos eixos principais, e orientadas para o centro da cena. Uma destas câmaras deve utilizar projecção ortogonal e a outra projecção em perspectiva. Adicionar também uma câmara móvel com projecção em perspectiva, posicionada na aresta superior do smartwatch que deve apontar para diante (i.e., de forma não perpendicular ao ecrã do relógio). Deve ser possível alternar entre as seis câmaras utilizando as teclas numéricas: '1' (topo); '2' (lateral); '3' (frontal); '4' (câmara fixa com projecção ortogonal); '5' (câmara fixa com projecção em perspectiva); '6' (câmara móvel com projecção em perspectiva). Para cada uma das câmaras, adicionar um *CameraHelper* do *Three.js*, de modo a facilitar o *debugging* visual. A visibilidade deste *CameraHelper* deve poder ser alternada através da tecla 'H' ('h').
2. **[1,00 valores]** Modelar em *Three.js* uma versão simplificada do *DroneWatch*, tomando como referência a Figura 1. Note-se que não se pretende uma cópia fiel e integral do que é representado na Figura 1; o resultado deverá consistir, pelo menos, numa aproximação *low-poly*. Sendo um objecto articulável, é necessário definir uma hierarquia de transformações entre as diferentes peças que o compõem, de modo a permitir o movimento dos rotores. As peças a considerar são as seguintes: (i) quatro caixilharias de rotores; (ii) mecanismos e articulações que permitem que a secção de cada rotor saia da estrutura do smartwatch; (iii) as respectivas hélices; (iv) estrutura do smartwatch; (v) câmara incorporada no bordo superior do smartwatch; (vi) botão largo que despoleta a descolagem do dispositivo, colocado no bordo inferior do smartwatch; (vii) pulseira do relógio. Com excepção da pulseira, que pode ser importada de um repositório online de malhas 3D, as restantes peças devem ser modeladas recorrendo, exclusivamente, a pelo menos uma das seguintes primitivas geométricas disponíveis na biblioteca *Three.js*^{1,2}: cilindros, cubos e toros. Para cada um dos objectos, adicionar um *AxesHelper* do *Three.js*, de modo a facilitar o *debugging* visual. A visibilidade deste *AxesHelper* deve poder ser alternada através da tecla 'H' ('h').

Nota: O dimensionamento dos objectos deve ser definido de forma plausível, tendo em conta o tamanho médio de um smartwatch de formato quadrado.

3. **[0,50 valores]** Modelar em *Three.js* uma versão simplificada de um conjunto de balões vermelhos (pelo menos 4 balões). Mais uma vez pretende-se uma representação *low-poly*. Cada balão deve ser modelado com o seguinte conjunto de peças: (i) corpo do balão; (ii) nó ou extremidade

¹ Vários exemplos podem ser encontrados na documentação oficial <https://threejs.org/manual/#en/primitives>

² Ilustração com várias primitivas *Three.js* <https://threejs.org/manual/examples/primitives.html>

inferior do balão; e (iii) fita suspensa. A modelação dos balões deve recorrer exclusivamente às seguintes primitivas geométricas disponíveis na biblioteca Three.js: esferas, cones e cilindros. Os balões devem ser colocados aleatoriamente no espaço, a uma cota superior à pulseira do *DroneWatch*. Para cada um dos objectos, adicionar um *AxesHelper* do *Three.js*, de modo a facilitar o debugging visual. Este *AxesHelper* deve poder ser alternado entre visível e invisível, premindo a tecla H.

Nota: O dimensionamento das objectos deve ser definido de forma plausível, respeitando proporções consistentes entre o corpo do balão, o nó e a fita.

4. **[0,50 valores]** Permitir ao utilizador controlar o “*DroneWatch*” permitindo a sua descolagem e aterragem a partir da plataforma da pulseira, bem como voar pela cena para rebentar os balões. O controlo do movimento deve ser feito recorrendo às seguintes teclas:
 1. ‘Q(q)’ — botão *toggle* para accionar a extensão ou recolha dos rotores, para fora ou para dentro da estrutura do *smartwatch*, respectivamente; assim que os rotores estiverem em extensão total, as hélices são accionadas girando com uma velocidade de rotação constante;
 2. ‘A(a)’ e ‘D(d)’ — controlar o deslocamento δx , que translada o “*DroneWatch*”;
 3. ‘W(w)’ e ‘S(s)’ — controlar o deslocamento δy , que translada o “*DroneWatch*”;
 4. ‘U(u)’ e ‘J(j)’ — controlar o deslocamento δz , que translada o “*DroneWatch*”;
 5. ‘I(i)’ e ‘K(k)’ — controlar o ângulo θ_1 , correspondente à rotação em torno do eixo yaw do “*DroneWatch*”;
 6. ‘O(o)’ e ‘L(l)’ — controlar o ângulo θ_2 , correspondente à rotação em torno do eixo pitch do “*DroneWatch*”.

Nota: Apenas é possível movimentar o “*DroneWatch*” quando os rotores se encontram completamente na sua posição final.

Nota: O movimento do objecto articulado deve ocorrer a velocidade constante, sendo a direcção determinada por um vector tridimensional. O cálculo do movimento deve ter em consideração que o utilizador pode pressionar várias teclas em simultâneo.

Nota: Cada grau de liberdade deve possuir uma amplitude de movimento limitada a um intervalo de valores pré-definido, de modo a evitar poses cinematicamente impossíveis aquando da extensão e recolha dos rotores.

5. **[0,10 valores]** A representação visual dos objectos da cena deve poder ser alternada entre modelo de arames (*wireframe*) e representação sólida, utilizando a tecla ‘7’.
6. **[0,75 valores]** Implementar a detecção de colisões entre o “*DroneWatch*” e os balões. A detecção deve recorrer a pares de contacto esfera–esfera, isto é, cada balão e rotor deve possuir geometrias de colisão na forma de esferas, solidárias e alinhadas com o respectivo referencial local. Quando detetada a colisão, deve ser posta em marcha uma animação que termina com o desaparecimento do balão da cena. Durante o processamento desta animação resultante da colisão, nenhuma tecla deverá ser processada.

7. **[0,40 valores]** Incluir um *Heads-Up Display* (HUD) com o mapa das teclas disponíveis. O HUD deve indicar quais as teclas actualmente activas. Deve ainda ser incluído um contador de FPS (e.g., stats.js) e um painel de controlo para manipular parâmetros do grafo de cena (e.g., dat.GUI).
8. **[0,50 valores]** Exportar o modelo do “*DroneWatch*” para o formato glTF e importá-lo para o *Lens Studio*³ (SDK do *Snapchat*). O objectivo consiste em criar uma lente de Realidade Aumentada (‘*lens*’) simples que permita visualizar o “*DroneWatch*” directamente sobre o pulso do utilizador. Para tal, para além do *Lens Studio*, que deverá ser instalado nos computadores pessoais, será também necessário instalar a aplicação *Snapchat*⁴ nos *smartphones*, de modo a testar e visualizar a lente criada.

Mais notas importantes ...:

1. A biblioteca Three.js já disponibiliza as classes principais necessárias para o desenvolvimento dos projectos desta cadeira. Assim, recomenda-se que os alunos adoptem uma abordagem de programação orientada a objectos, recorrendo às classes disponibilizadas por esta biblioteca. Devem igualmente seguir boas práticas de programação, de modo a promover a reutilização de código em entregas posteriores e a facilitar a escalabilidade das soluções.
2. Dado que os projectos desta cadeira são de pequena a média complexidade, não se justifica que a organização de pastas seja desnecessariamente complexa ou desordenada. O código-fonte deverá, em princípio, resumir-se a um único script, devendo ser respeitada a estrutura definida no template disponibilizado para o efeito (Template – Trabalhos B e C). O incumprimento desta estrutura implicará uma pequena penalização.
3. Não devem ser utilizadas bibliotecas externas, nem funções do Three.js destinadas à detecção de colisões ou à implementação da física do movimento. Pretende-se que essas funcionalidades sejam implementadas directamente pelos alunos.
4. Para além dos acontecimentos de *update* e *display*, existem outros acontecimentos relevantes, tais como teclas pressionadas ou libertadas, temporizadores e redimensionamento da janela. Recomenda-se que estes acontecimentos sejam tratados de forma independente, recorrendo às respectivas funções de callback.
5. Note-se que, neste trabalho, não é requerida a implementação do tratamento do redimensionamento da janela; no entanto, esta funcionalidade será solicitada no **Trabalho C**.
6. A implementação dos trabalhos desenvolvidos nos laboratórios de Computação Gráfica deve seguir o ciclo de animação (*update/display cycle*). Este padrão de desenho, comum em aplicações de computação gráfica interactiva, separa o processo de actualização do estado da aplicação da renderização da cena em duas fases distintas. Na fase de *display*, são tipicamente executados três passos: limpar o *buffer*, desenhar a cena e forçar o processamento dos comandos de renderização. Na fase de *update*, o estado dos objectos da cena é actualizado de acordo com a física ou lógica da aplicação. É também nesta fase que devem ser efectuadas a detecção de colisões e a implementação dos comportamentos associados.

³ Lens Studio – <https://developers.snap.com/lens-studio/home>

⁴ SnapChat – <https://www.snapchat.com/download>